data_loading.ipynb — team walkthrough

What the notebook builds, cell by cell, and what still needs to happen before modeling (train/test split).

---

1. What dataset is this

This is the **NASA IMS bearing run-to-failure dataset** — not the fault-classification dataset (HH/IR/OR) from earlier. Four bearings run continuously on a shaft rig until one or more fail. Every few minutes the rig records a short vibration snapshot; each snapshot is saved as its own text file, and the filename itself encodes the timestamp (e.g. `2004.02.12.10.32.39`). There are three separate experiments ("1st_test", "2nd_test", "3rd_test"), each with its own folder of snapshot files.

Two structural quirks matter for everything downstream:

- **Channel layout differs by test.** Test 1 recorded 2 accelerometer channels per bearing (8 columns total); tests 2 and 3 recorded only 1 channel per bearing (4 columns total). That's why `CHANNEL_MAP` has different column indices per test.
- **Not every bearing actually fails.** Each test only has one or two bearings that degrade to failure; the rest stay healthy for the whole run. `FAILED_BEARINGS` restricts all downstream processing to just the bearings known (from prior analysis of this dataset) to have actually failed.

Because this is a **run-to-failure** dataset rather than a labeled-class dataset, the goal is different from the HH/IR/OR classification notebook: instead of predicting which fault type a signal shows, this pipeline builds a target for **Remaining Useful Life (RUL)** — how much time is left before that bearing fails — using each recording's timestamp relative to the last recording in that bearing's life.

---

2. Cell-by-cell walkthrough

Semantics = what it accomplishes and why. Syntax notes = the Python/pandas/numpy/scipy mechanics worth understanding.

Cell	Key code	What it does (semantics)	Syntax notes
1 Imports	`from pathlib import Path import numpy as np, pandas as pd from scipy.stats import skew, kurtosis from scipy.signal import welch`	Pulls in everything used later: array math (numpy), tabular data (pandas), plotting (matplotlib), and the two scipy functions that do the statistical heavy lifting (skew/kurtosis, and Welch's method for power spectral density).	`from scipy.stats import skew, kurtosis` imports the functions directly, so later code calls `skew(x)` rather than `stats.skew(x)`.
2 Dataset path check	`DATASET_PATH = Path("IMS") if not DATASET_PATH.exists(): raise FileNotFoundError(...)`	Fails loudly and immediately if the IMS folder isn't where expected, instead of failing confusingly many cells later.	`Path` objects (not raw strings) are used throughout for file paths — this is why `.exists()`, `.iterdir()`, `.resolve()` work directly.
3 Count files per test	`for folder in sorted(DATASET_PATH.iterdir()): if folder.is_dir(): ...`	Quick sanity check: how many recordings exist in each test folder, and what are the first/last filenames (a cheap way to eyeball the time range covered).	`iterdir()` is not guaranteed sorted by itself, hence the outer `sorted(...)`. Note: this cell's execution number (In[13]) is out of order relative to the notebook's physical position — a sign it was re-run out of sequence during development. Doesn't affect correctness, but worth tidying before sharing.

Cell	Key code	What it does (semantics)	Syntax notes

4 get_recording_files()	`def get_recording_files(test_folder):`
	`  files = [p for p in test_folder.rglob("*") if p.is_file()]`
	`  return sorted(files)`
		Reusable helper: lists every recording file inside a test folder, ignoring subdirectories, and returns them sorted.	`rglob("*")` recurses into subfolders (glob doesn't). Sorting here works only because filenames are timestamp strings in `YYYY.MM.DD.HH.MM.SS` format — lexicographic string sort happens to equal chronological order for that format specifically.

5 Load one recording (demo)	`sample = pd.read_csv(sample_file, sep=r"\s+", header=None, engine="python")`	Loads a single snapshot file to inspect its shape before writing the full loop — standard 'test on one, then scale up' practice.	`sep=r"\s+"` is a regex meaning 'one or more whitespace characters' — these files are whitespace-delimited, not comma-delimited, hence `engine="python"` (the fast C parser doesn't support regex separators). `header=None` because the raw files have no column names.
6 Inspect structure	`print(sample.shape);` `print(sample.dtypes)`	Confirms row/column counts and that every column parsed as numeric (not accidentally read as text).	Straightforward pandas attribute access — no computation.
7 CHANNEL_MAP	`CHANNEL_MAP = {"1st_test": {"bearing_1": 0, ...}, ...}`	Maps bearing name → column index, per test. Test 1 has 8 columns (2 per bearing) so bearings map to columns 0, 2, 4, 6 (every other column — likely the X-axis channel, skipping Y). Tests 2/3 have 4 columns (1 per bearing) so they map directly to 0, 1, 2, 3.	Nested dict literal. No computation — this is a lookup table encoding domain knowledge about the file format that isn't discoverable from the data itself.
8 FAILED_BEARINGS	`FAILED_BEARINGS = {"1st_test": ["bearing_3","bearing_4"], ...}`	Another lookup table, this time recording which bearings in each test actually degraded to failure (known from prior analysis of this public dataset, not derivable from a quick glance at the files). Everything from here on only processes these bearings.	Plain dict of lists.
9 parse_timestamp()	`def parse_timestamp(file_path):` `  return pd.to_datetime(file_path.name, format="%Y.%m.%d.%H.%M.%S", errors="coerce")`	Converts a filename string into an actual datetime object, so recordings can be ordered and time-differenced later (needed for the RUL calculation).	`errors="coerce"` means any filename that doesn't match the format returns `NaT` instead of raising — silently produces missing timestamps rather than crashing, worth being aware of if any files are misnamed.

10 create_windows()	`n_windows = len(signal) // window_size`
	`trimmed = signal[:n_windows*window_size]`
	`return trimmed.reshape(n_windows, window_size)`
		Slices one file's raw signal into fixed-length, **non-overlapping** windows (1024 samples each). This windows *within a single snapshot file* — different from splitting the whole life-span signal; each file is only ~20,480 samples (~1 second), so this just chops that one second into ~20 chunks.	Integer floor division (`//`) computes how many full windows fit; any leftover samples that don't fill a complete window are dropped (`trimmed_signal`). `reshape(n_windows, window_size)` is a zero-overlap, zero-copy-when-possible way to window a 1D array — much faster than a Python loop.

Cell	Key code	What it does (semantics)	Syntax notes
11 Apply windowing (demo)	`signal = sample.iloc[:, CHANNEL_MAP["1st_test"]["bearing_3"]].values windows = create_windows(signal, window_size=1024)`	Runs the windowing function on one real signal (bearing_3, first file of test 1) to confirm shapes look right before using it in the full loop.	`.iloc[:, col_index]` selects a column by position (not by name, since there are no column names). `.values` converts the pandas Series to a raw numpy array.
12 Plot 3 windows	`fig, axes = plt.subplots(3, 1, ...) for i, ax in enumerate(axes): ax.plot(windows[i])`	Visual sanity check — do the windowed chunks look like real vibration waveforms, not garbage from a reshape mistake?	Standard matplotlib subplot loop.
13 extract_time_features()	`rms = sqrt(mean(x**2)); peak = max(abs(x)); ... returns dict of 13 stats`	Computes 13 time-domain statistics per window: mean, std, variance, RMS, peak, peak-to-peak, skewness, kurtosis, crest factor, shape factor, impulse factor, energy, mean-absolute. These summarize the shape/intensity of one window's waveform as single numbers.	Returns a plain Python `dict` (not a pandas object) — this is deliberate, since dicts are cheap to accumulate in a list and convert to a DataFrame once at the end, rather than growing a DataFrame row-by-row (which is much slower in pandas).
14 extract_frequency_features() *(defined twice — cells 13b & 14 in dupe form)*	`frequencies, psd = welch(x, fs=fs, nperseg=min(256, len(x))) dominant_frequency = frequencies[argmax(psd)] ... returns dict of 6 stats`	Computes 6 frequency-domain statistics using Welch's method (a smoothed periodogram, more stable than a raw FFT for noisy vibration signals): dominant frequency, spectral centroid, spectral bandwidth, spectral entropy, spectral energy, and max PSD value.	`nperseg=min(256, len(x))` caps the Welch segment length at 256 samples (or the signal length if shorter) — this sets the frequency resolution of the estimate. **Note:** this exact function is defined twice in the notebook, identically, in two separate cells — harmless (the second definition just silently overwrites the first) but worth deleting the duplicate before sharing.
15 extract_features()	`return {**time_features, **frequency_features}`	Combines the time-domain and frequency-domain feature dicts into one dict per window — this is the single function the main loop actually calls per window.	`{**a, **b}` is dict-unpacking merge syntax — creates a new dict containing all key-value pairs from both, since the two source dicts don't share any keys this is a safe merge.
16 Main feature-extraction loop	`for test_name, failed_bearings in FAILED_BEARINGS.items(): for file_path in recording_files (sorted by timestamp): for bearing_name in failed_bearings: windows = create_windows(signal) for window in windows: extract_features(...) → append row`	The core of the notebook. For every failing bearing, in every test, in every recording file (in chronological order), the signal is windowed and every window becomes one row of features, tagged with `test_set`, `bearing`, `recording_index` (the file's position in time order), `window_index` (position within that file), and `timestamp`. The result, `features_df`, has **one row per window across each failing bearing's entire life**.	Triple-nested loop; rows are collected into a plain Python list (`all_rows`) and converted to a DataFrame only once at the very end (`pd.DataFrame(all_rows)`) — the efficient pandas pattern mentioned in cell 13.
17 Re-demo extract_features	`features = extract_features(windows[0]) pd.DataFrame([features])`	A leftover exploratory cell — reuses the `windows` variable from cell 11 (bearing_3, first file of test 1 only), unrelated to the full `features_df` built in cell 16. Just a display check, not part of the pipeline.	**Careful:** the variable name `windows` is reused/shadowed between cell 11's single-file demo and this cell — it does not refer to anything from the big loop. Easy to misread when skimming.

Cell	Key code	What it does (semantics)	Syntax notes
18 Sort chronologically	`features_df = features_df.sort_values(by=["test_set","bearing","timestamp","window_index"]).reset_index(drop=True)`	Puts every bearing's rows in strict time order. Necessary before computing anything that depends on 'how far this row is from the end of that bearing's life'.	`reset_index(drop=True)` renumbers the DataFrame index 0..N after sorting, discarding the old (now-scrambled) index rather than keeping it as a column.
19-20 RUL (Remaining Useful Life)	`failure_timestamp = groupby(["test_set","bearing"])["timestamp"].transform("max")` `RUL_hours = (failure_timestamp - timestamp) / 3600`	**This defines the regression target.** For each bearing, the last recorded timestamp is treated as the failure point. Every row's RUL is how many hours remain until that bearing's last recording, computed from that row's own timestamp.	`.transform("max")` broadcasts the group's max timestamp back onto every row of that group (unlike `.agg`, which would collapse to one row per group) — this is what lets the subtraction happen row-wise.

21-24 RUL_fraction, health_index, failure_risk_score	`RUL_fraction = (RUL_hours / group max RUL_hours).clip(0,1)`
	`health_index = RUL_fraction * 100`
	`failure_risk_score = (100 - health_index).clip(0,100)`
		Normalizes RUL into a 0–100 'health score' (100 = start of life, 0 = failure) and its inverse 'risk score' (0 = healthy, 100 = about to fail) — easier to interpret and plot than raw hours, and comparable across bearings with different total lifespans.	`.clip(0,1)` / `.clip(0,100)` guards against floating-point rounding pushing values slightly outside the intended range.

25 Save to CSV	`features_df.to_csv("ims_features_dataset.csv", index=False)`	Persists the full engineered dataset — **every window from every failing bearing's entire life, in one file, still unsplit.** See Section 3 for why this matters.	`index=False` omits writing the pandas row-number index as a spurious extra column.
26-35 Visualization cells	`seaborn/matplotlib: window counts per bearing, early-vs-late waveform, feature degradation trends vs RUL_hours, health/risk trend lines, life-stage KDE distributions, correlation heatmap, boxplots by bearing, RMS-vs-kurtosis scatter, summary stats table`	Exploratory data analysis on the full (still-unsplit) dataset: confirms that features actually trend with degradation (e.g. RMS and kurtosis should rise as a bearing nears failure) and that different bearings show visibly different feature distributions — the qualitative justification for why these features should be predictive.	Standard seaborn/matplotlib calls; each trend plot groups by `RUL_hours` and inverts the x-axis so 'time until failure' reads left (far) to right (zero) chronologically.

3. Train/test split — what happens next

Bottom line: this notebook does not split the data into train/test anywhere. It stops at building and saving `ims_features_dataset.csv` (cell 25) and then only visualizes the full, unsplit dataset. A split needs to be added as a deliberate next step before any model gets trained on this file.

Why this matters: what kind of dataset this actually is

Every row in `features_df` is one 1024-sample window, and consecutive windows — whether from the same file or the very next file a few minutes later — come from a bearing that is barely different from one moment to the next. This is the same leakage concern discussed for the classification notebook, just in a time-series-regression form: **if windows are shuffled randomly across train/test, the model will be tested on windows that are near-duplicates of windows it trained on** (same bearing, nearly the same point in its degradation curve). Reported accuracy would look great and would not reflect real generalization.

The correct way to split this dataset

- **Split by whole bearing run, not by window.** Each (test_set, bearing) combination is one continuous life history. The safest split holds out one or more entire bearing runs for testing and trains on the rest — e.g. train on 1st_test/bearing_3 and 2nd_test/bearing_1, test on 1st_test/bearing_4 and 3rd_test/bearing_3. This tests whether the model generalizes to a bearing it has never seen degrade, which is the real-world use case.
- **If you need a split within a single bearing's life** (e.g. to test 'can this model predict the ending given the beginning'), split chronologically: an early contiguous portion of that bearing's timeline as train, a later portion as test — never randomly shuffled, and ideally with a time gap between them, following the same gap logic used in the fault-classification notebook.
- **Either way, group by (test_set, bearing) before splitting** — use something like `features_df.groupby(["test_set","bearing"])` to reason about the split at the bearing level, then filter `features_df` down to train/test subsets by that grouping, not by a random row sample.

Suggested next cell (for whoever picks this up)

```
TEST_BEARINGS = [("1st_test", "bearing_4"), ("3rd_test", "bearing_3")]
is_test = features_df.set_index(["test_set","bearing"]).index.isin(TEST_BEARINGS)
train_df = features_df[~is_test].reset_index(drop=True)
test_df = features_df[is_test].reset_index(drop=True)
```

Open question for the team

Which held-out bearing(s) to use for testing is a modeling decision, not just a coding one — worth discussing before it's added: do we want a single held-out bearing (simple, but only one test example per test run), or a proper cross-validation scheme that rotates through each failing bearing as the test set in turn (more robust, more compute)? Given there are only 4 failing bearings total across all three tests, a leave-one-bearing-out cross-validation is probably more informative than a single fixed split.